人脸识别 SDK
及 DEMO 使用说明

V10

版权所有 翻版必究

2021 年 1 月

目　录

一.　SDK 概述 3
二.　SDK 接口说明 4
1．　概述 4
2．　人脸属性及质量 4
3．　获取 SDK 版本 5
4．　SDK 初始化 5
5．　SDK 反初始化 5
6．　检测人脸 6
7．　检测人脸质量级别 6
8．　获取特征码大小 6
9．　提取特征码 7
10．　一对一特征比对 7
11．　创建一对多特征比对列表 7
12．　向一对多特征比对列表加入目标特征 7
13．　从一对多特征比对列表中删除部分特征 8
14．　清空一对多特征比对列表中的所有特征 8
15．　一对多特征比对 8
16．　销毁一对多特征比对列表 9
17．　活体检测 9
18．　授权接口 10
19．　辅助接口 11
三.　其它 12

人脸识别 SDK 使用说明

一. SDK 概述

为方便使用本公司的人脸识别 SDK，我们除提供 SDK 及使用说明外，还可提供 DEMO 可执行程序及 DEMO 源代码。

A． SDK 包含人脸识别 SDK 所需库文件及头文件，有 32 位 CPU 版本、64 位 CPU 版本，64 位 CPU+GPU 版本，各版本又分单通道版和多通道版本。

32 位 CPU 版本为 32 位动态库，只支持 32 位应用程序或 32 位容器调用。可在 32 位系统或 64 位系统上运行。

64 位 CPU 版本和 64 位 CPU+GPU 版本均为 64 位动态库，只支持 64 位应用程序或 64 位容器调用。只能在 64 位系统上运行。

64 位 CPU+GPU 版本还需要安装 GPU 运行环境才能运行。

多通道版本各功能均支持多线程并行，但要注意多线程调用相同功能且指定的通道号也相同时，各线程也是串行工作的，执行同一功能的每个线程使用自己唯一的通道号才会并行工作才能发挥出处理器的性能；单通道版本同一个功能同一时刻只能有一个线程执行，多线程调用同一功能也是串行方式工作。

各版本接口一致（基本相同），多通道版本比单通道版本多一个通道参数。下面以单通道版本为例说明。

B． SDK 主要包含三个文件：算法核心文件 IdFaceSdk.dll、头文件 IdFaceSdk.h 及库文件 IdFaceSdk.lib。头文件及库文件都是辅助开发所用，发布版本只需要核心文件：

IdFaceSdk.dll ---- SDK 核心动态库

IdFaceSdk.h ---- SDK 接口头文件（VC/VS 开发工具直接引用，其它开发工具仅供参考）

IdFaceSdk.lib ---- SDK 静态链接库（VC/VS 开发工具可用，其它开发工具请动态加载）

C． DEMO 可执行程序为使用本 SDK 的 DEMO，包括人证核验 FaceVerify 和重点人员稽查 FaceRecognition 及活体检测 FaceLive。每个 DEMO 都必须连接本公司提供的 USB 加密狗才能使用，必须连接摄像头才能使用。FaceVerify 为读取身份证信息并调用本 SDK 与从摄像头抓取的人脸进行人脸识别的演示程序，必须连接身份证阅读器，支持华视、国腾、精伦等身份证阅读器，支持笔记本电脑内置的摄像头，也支持外接的 USB 摄像头（笔记本电脑使用外接 USB 摄像头时，请先禁用内置摄像头）。FaceRecognition 为通过摄像头或批量导入照片文件进行注册，并通过摄

像头进行一对多对的演示程序。FaceLive 目前支持彩色+红外双目相机双路视频活体检测，也支持单红外视频检测；即将支持单彩色视频检测

D．DEMO 源代码为上述 DEMO 程序的 VC/VS 源代码，供开发人脸识别应用程序的程序员参考，VC/VS 编译器可直接编译运行，其它开发工具请参照源代码自行调用。

二. SDK 接口说明

1. 概述

本 SDK 支持人脸属性检测，包括人脸坐标、眼睛坐标、人脸偏转角度等，根据实际应用要求，可以对最小人脸、最大偏转角度等做进一步限制，并且用文字结合语言等方式提示用户调整好姿态。

特征比对支持一对一比对与一对多比对。两个特征直接比对时，如人脸识别，请调用一对一接口，一对一接口更简单方便，但如果一个特征要与多个特征进行一对多比对时，当参与比对的特征数量较小时，如与同一个人的多个模板进行比对，也可以循环调用一对一比对接口。当参与比对的特征数量较大时，需要调用一对多比对接口，一对多比对接口较为复杂，但特征数量较大时，比对效率远高于一对一比对。

2. 人脸属性及质量

人脸位置属性包括人脸坐标、眼睛、嘴巴、鼻子坐标及其它几何特征。是人脸检测返回的结果，也是特征提取要输入的参数。

人脸属性定义如下：

```
typedef struct FACE_DETECT_RESULT {
    RECT rcFace; // 人脸坐标
    POINT ptLeftEye, ptRightEye; // 人眼坐标
    POINT ptMouth, ptNose; // 嘴巴和鼻子坐标
    int nAngleYaw, nAnglePitch, nAngleRoll; // 人脸角度
    int nQuality; // 人脸质量（保留）
    char FaceData[512]; // 人脸详细几何特征
} FACE_DETECT_RESULT, *LPFACE_DETECT_RESULT;
```

人脸质量级别包括人脸完整性、人脸大小、姿态、亮度、遮罩、光源方向等质量属性的级别。是人脸质量级别检测返回的结果。

```
typedef struct FACE_QUALITY_LEVEL_EX {
    int nHalf; // 人脸完整性: 0-正常，1-人脸不太完整，2-人脸非常不完整
    int nSmall; // 人脸大小：0-正常，1-人脸较小，2-人脸太小
    int nPosture; // 姿态：0-正常，1-偏头较多，2-偏头太多
```

int nMask; // 脸部遮挡: 0-正常，1-人脸有遮挡，2-人脸遮挡太多

int nFaceMask; // 口罩：0-正常，1-戴口罩未遮住鼻子，2-戴口罩已遮住鼻子

int nHat; // 帽子：0-正常，1-有戴帽，2-帽子遮挡脸部

int nGlasses; // 眼镜: 0-正常，1-有戴眼镜，2-确认戴眼镜

int nGape; // 张嘴程度(0-100)

int nLeftEyeStatus, nRightEyeStatus; // 睁眼程度(0-100)

int nBlur; // 模糊度：0-正常，1-较模糊，2-太模糊

int nBright; // 脸部曝光度：0-正常，1-太暗，2-过爆

int nLight; // 光源方向: 0-正常，1-侧光，2-顶光, 3-逆光

} FACE_QUALITY_LEVEL_EX, *LPFACE_QUALITY_LEVEL_EX;

3. 获取 SDK 版本

获取 SDK 版本接口函数为 IdFaceSdkVer，定义如下：

int IdFaceSdkVer(void);

本接口无参数，调用后返回 SDK 版本号。

备注：本接口可不调用，也可在任何时候调用

IDFACESDK_API int IdFaceSdkVer(void);

4. SDK 初始化

SDK 初始化接口为 IdFaceSdkInit，定义如下：

int IdFaceSdkInit(void);

本接口无参数，调用成功返回 0，未授权返回-1，算法初始化失败返回-2

返回-1 时请确认插入了本公司的 USB 加密狗，且确认该加密狗是永久有效的或者未超出有效期。

返回-2 时请确认本文档所描述的依赖文件都放在应用程序的工作目录下。

备注：附获取 SDK 版本外的任何接口都必须在 SDK 初始化成功后才能调用。

5. SDK 反初始化

SDK 反初始化接口为 IdFaceSdkUninit，定义如下：

void IdFaceSdkUninit(void);

本接口无参数，也无返回值。

如果 SDK 初始化成功，则当应用程序退出前，应调用本接口正常释放资源。

备注：必须在初始化成功后调用，反初始化后不能再调用除获取 SDK 版本及 SDK 初始化外的任何接口。

6. 检测人脸

检测人脸接口为 IdFaceSdkDetectFace，定义如下：

int IdFaceSdkDetectFace(BYTE* pRgb24, int nWidth, int nHeight, LPFACE_DETECT_RESULT pFaceResult);

输入参数定义如下：

pRgb24 ---- RGB24 格式的图象数据

nWidth ---- 图象数据宽度（象素单位）

nHeight ---- 图象数据高度（象素单位）

输出参数定义如下：

pResultBuf ---- 检测到的人脸属性（人脸及眼睛等坐标位置及角度等，调用前必须分配有效的空间）

返回 1 表示检测到人脸，0 表示无人脸，-1 表示检测失败

7. 检测人脸质量级别

检测人脸质量级别接口为 IdFaceSdkFaceQualityLevelEx，定义如下：

int IdFaceSdkFaceQualityLevelEx (BYTE* pRgb24, int nWidth, int nHeight, LPFACE_DETECT_RESULT pFaceResult, LPFACE_QUALITY_LEVEL_EX pFaceQualityLevel);

输入参数定义如下：

pRgb24 ---- RGB24 格式的图象数据

nWidth ---- 图象数据宽度（象素单位）

nHeight ---- 图象数据高度（象素单位）

pFaceResult ---- 人脸检测返回的人脸坐标属性

输出参数定义如下：

pFaceQualityLevel ---- 检测到的人脸质量级别属性（包括人脸大小、姿态、亮度、遮罩、光源方向等质量级别，调用前必须分配有效的空间）

返回 0 表示成功，-1 表示检测失败

8. 获取特征码大小

获取特征码大小接口为 IdFaceSdkFeatureSize，定义如下：

int IdFaceSdkFeatureSize(void);

本接口无参数，调用后返回特征码大小。

9. 提取特征码

提取特征码接口为 IdFaceSdkFeatureGet，定义如下：

int IdFaceSdkFeatureGet(BYTE* pRgb24, int nWidth, int nHeight, LPFACE_DETECT_RESULT pFaceResult, BYTE* pFeature);

输入参数定义如下：

pRgb24 ---- RGB24 格式的图象数据

nWidth ---- 图象数据宽度

nHeight ---- 图象数据高度

pResultBuf ---- 人脸检测返回的人脸属性（必须将检测人脸返回的结果原样传入）

输出参数：人脸特征码（调用前必须分配有效的空间）

返回 0 表示成功，返回-1 表示失败

10. 一对一特征比对

两张人脸的特征比对接口为 IdFaceSdkFeatureCompare，定义如下：

IDFACESDK_API unsigned char IdFaceSdkFeatureCompare(BYTE* pFeature1, BYTE* pFeature2);

输入参数：

pFeature1 ---- 第 1 张人脸的特征

pFeature2 ---- 第 2 张人脸的特征

返回：返回两个人脸特征对应的人脸的相似度（0-100）

11. 创建一对多特征比对列表

创建一个一对多特征比对列表的接口为 IdFaceSdkListCreate，定义如下：

IDFACESDK_API HANDLE IdFaceSdkListCreate(int nMaxFeatureNum);

输入参数：nMaxFeatureNum ---- 列表能容纳的最大特征数

返回：返回特征比对列表句柄，为空表示创建列表失败

备注：能容纳的最大特征数越大，则消耗的内存越多，另外实际支持的最大特征数可能受加密狗限制

12. 向一对多特征比对列表加入目标特征

向一对多特征比对列表加入目标特征的接口为 IdFaceSdkListInsert，定义如下：

IDFACESDK_API int IdFaceSdkListInsert(HANDLE hList, int* pnPos, BYTE* pFeatures, int nFeatureNum);

输入参数：

hList ---- 目标特征要加入的特征比对列表句柄

pnPos ---- 指针非空时存放特征插入位置，位置值为 0 表示插到列表最前面，为-1 或大于当前列表中的特征数则表示插到列表最后面

pFeatures ---- 要插入的目标特征码（支持一次插入多个特征码），多个特征码时按特征码大小（IdFaceSdkFeatureSize 函数接口返回的值）对齐进行排列

nFeatureNum ---- 要插入的特征数量

输出参数：pnPos ---- 指针非空时返回第一个特征的插入位置

返回：返回当前列表的总特征数

13. 从一对多特征比对列表中删除部分特征

从一对多特征比对列表中删除部分特征的接口为 IdFaceSdkListRemove，定义如下：

IDFACESDK_API int IdFaceSdkListRemove(HANDLE hList, int nPos, int nFeatureNum);

输入参数：

hList ---- 要删除特征的特征比对列表句柄

nPos ---- 要删除的特征的起始位置

nFeatureNum ---- 要删除的特征数量

返回：返回当前列表的总特征数

14. 清空一对多特征比对列表中的所有特征

清空一对多特征比对列表中的所有特征的接口为 IdFaceSdkListClearAll，定义如下：

IDFACESDK_API void IdFaceSdkListClearAll(HANDLE hList);

输入参数：

hList ---- 要清空的一对多特征比对列表句柄

返回：无

备注：调用后列表中的实际特征数量为 0（列表能容纳的最大特征数保持不变）

15. 一对多特征比对

一对多特征比对的接口为 IdFaceSdkListCompare，定义如下：

IDFACESDK_API int IdFaceSdkListCompare(HANDLE hList, BYTE* pFeature, int nPosBegin, int nCompareNum, BYTE* pnScores);

输入参数：

hList ---- 存放要参与比较的目标特征库的特征比对列表句柄

pFeature ---- 要参与特征比对的源特征码

nPosBegin ---- 要参与比对的目标特征的起始位置（如果比对全部目标特征，则起始位置填 0）

nCompareNum ---- 要参与比对的目标特征数量（如果比对全部目标特征，则目标特征数量填 0 或-1 或实际特征数或最大特征数）

输出参数：pnScores ---- 顺序存放与各目标特征进行比对的相似度（每个特征比对的结果均为一字节，值范围为 0-100）

返回：返回实际参与比对的特征数量，也是返回的相似度个数

16. 销毁一对多特征比对列表

销毁一对多特征比对列表的接口为 IdFaceSdkListDestory，定义如下：

IDFACESDK_API void IdFaceSdkListDestory(HANDLE hList);

输入参数：

hList ---- 要销毁的一对多特征比对列表句柄

返回：无

备注：特征比对列表不再使用时需调用此接口释放资源

17. 活体检测

活体检测支持单目活体检测（支持单路彩色图象检测活体，也支持单路红外图象检测活体），也支持双目活体检测（支持彩色+红外图象检测活体）。

活体检测接口定义如下：

IDFACESDK_API int IdFaceSdkLiveFaceDetect(int nWidth, int nHeight, BYTE* pColorRgb24, LPFACE_DETECT_RESULT pColorFaceResult, BYTE* pBwRgb24, LPFACE_DETECT_RESULT pBwFaceResult);

输入参数：

nWidth ---- 图象数据宽度（象素单位）

nHeight ---- 图象数据高度（象素单位）

pColorRgb24 ---- RGB24 格式的彩色图象数据

pColorFaceResult ---- 彩色图象检测出的人脸参数（调用 IdFaceSdkDetectFace 对彩色图象检测出的结果）

pBwRgb24 ---- RGB24 格式的黑白图象数据

pBwFaceResult ---- 黑白图象检测出的人脸参数（调用 IdFaceSdkDetectFace 对黑白图象检测出的结果）

输出参数：无

返回值：返回 1 表示已确认是活体，0 表示本帧不确认是活体

备注：

A. 先对图象进行人脸检测，如果检测不到人脸，则直接判为非活体（如显示屏播放的电子照片或录像，在红外视频中就检测不到人脸）。
B. 单目检测只传入对应的单路图象及其人脸参数进行活体检测（另一路图象参数传入 NULL）。双目检测需同时传入彩色图象和黑白图象及其人脸参数进行活体检测。
C. 过程中只要有一次确认是活体，则本次结果确认为活体；如超时仍无一次确认是活体，则本次结果确认为非活体

18. 授权接口

SDK 支持硬授权、本地软授权和云授权。

硬授权直接插入加密狗，调用 SDK 初始化就可以成功，不需要调用任何授权接口。

本地软授权通过调用获取设备运行码接口得到设备运行码，将运行码发送给 SDK 供应商换取注册码，将注册码保存到 license.dat 文件并放置在运行目录，调用 SDK 初始化就可以成功。

云授权通过联系 SDK 供应商在云授权服务器上开户并购买授权，得到获取云授权的用户名与密码，调用云授权校验接口即可申请到授权，然后再调用 SDK 初始化就可以成功。

18.1 获取设备运行码接口

获取设备运行码接口（本地软授权适用）定义如下：

int WINAPI IdFaceSdkGetRunCode(char* pStrRunCode);

输入参数：无

输出参数：pStrRunCode ---- 设备运行码

返回值：0: 成功，-1：失败

18.2 云授权校验接口

云授权校验接口（云授权适用）定义如下：

int WINAPI IdFaceSdkLicenseCheck(const char* pStrUser, const char* pStrPwd, const char* pStrLicensePath);

输入参数：

strUser---- 客户的用户名

strPwd----客户申请授权的密码

strLicensePath----授权文件路径或文件名，必须为一个有效的本地目录或文件名并具有可写权限，用于存放授权数据

输出参数：

无

返回值：

1 ---- 成功，指定位置已经存在授权文件

0 ---- 成功，成功申请到授权并写入指定位置

-1 ---- 失败，用户名或密码错误

-2 ---- 失败，无有效授权，请购买授权

-3 ---- 失败，连接云服务器错误，请稍后重试

-4 ---- 失败，其它错误

备注：

1. 必须在初始化前调用
2. 成功申请到授权时自动写入指定位置，如指定位置存在授权则不会重复申请
3. 每次传入的授权文件路径或文件名必须固定不变，否则因找不到原来的授权文件而重复申请授权导致的损失由调用方负责

19. 辅助接口

辅助接口主要有读取图象文件转换为 RGB24 格式的图象数据，支持 BMP、JPEG、PNG 文件。**其它辅助接口请参见头文件说明。**

读取图象文件接口定义如下：

int WINAPI ReadImageFile(const char* pFileName, BYTE* pRgbBuffer, int nBufSize, int* pnWidth, int* pnHeight, int nDepth);

输入参数：

pFileName ---- 图象文件名

nBufSize ---- 接收图象数据的缓冲区大小，当接收缓冲区大小为 0 或者接收缓冲区指针为空时，只输出图象的宽度

nDepth ---- 要接收的图象数据的位深度（需要灰度图象时填 8，需要 RGB24 图象时填 24）

输出参数：

pRgbBuffer ---- 接收图象数据，必须分配足够大的缓冲区并且将分配的缓冲区大小通过 nBufSize 参数传入

pnWidth ---- 接收图象宽度

pnHeight ---- 接收图象高度

返回值：

0 ---- 成功

-1 ---- 参数错误

-2 ---- 打开文件失败

其它 ---- 图象解析失败

备注：不知道图象分辨率因为不确认应分配的缓冲区大小时，可以先填入空缓冲区指针检测图象分辨率，再分配合适的缓冲区重新调用获得图象数据

三. 其它

我们可提供 Windows、Android、PC-Linux、嵌入式 LINUX 等各种平台的人脸识别、人证核验、活体检测 SDK 及 DEMO。

任何疑问请联系我们的销售或技术支持人员。

谢谢！